

Web2Compile: Uma Web IDE para Redes de Sensores sem Fio

Augusto Santos¹, Márcio Farias¹, Gabriel Rocha¹, Marina V. Pereira¹, Claudio M. de Farias¹,
Tiago C. França¹ e Rafael O. Costa¹

¹Departamento de Ciência da Computação - Universidade Federal do Rio de Janeiro
(UFRJ)
Rio de Janeiro - RJ - Brazil

Abstract. *This paper presents the Web2Compile WebIDE for Wireless Sensor Networks Application development using TinyOS and NesC. Using Web2Compile, an user is able to generate a sensor image and to simulate their behaviour through TOSSIM simulator only using an updated browser. Web2Compile intends to eliminate the need of complex install procedures required nowadays by Wireless Sensor Networks development environments.*

Resumo. *Este trabalho apresenta a WebIDE Web2Compile para o desenvolvimento de aplicações para redes de sensores sem fio usando TinyOS e NesC. Usando Web2Compile, um usuário é capaz de gerar uma imagem do sensor e simular o seu comportamento através do simulador TOSSIM usando apenas um navegador atualizado. Web2Compile pretende eliminar necessidade de procedimentos de instalação complexos utilizados atualmente por ambientes de desenvolvimento para Redes de Sensores sem fio.*

1. Introdução

Os recentes avanços na tecnologia de sistemas micro-eleto-mecânicos, nas comunicações sem fio e na eletrônica digital possibilitaram a construção de sensores de tamanho reduzido, os quais, em grande número, permitem monitorar variáveis físicas e ambientais como temperatura, umidade, níveis de ruído e movimento de objetos com elevado grau de precisão [Culler et al. 2004]. Ao conjunto desses dispositivos dá-se o nome Redes de Sensores sem Fio (RSSFs). Três necessidades para o crescimento das RSSFs são: (i) a produção de sensores em larga escala, reduzindo o seu custo, (ii) maiores investimentos no desenvolvimento tecnológico desses dispositivos, levando a novas melhorias e capacidades e (iii) a falta de mão de obra capaz de produzir aplicações para esses dispositivos [Loureiro et al. 2003].

Apesar de ser um campo promissor, existem algumas dificuldades para o ensino de desenvolvimento de aplicações para RSSF. Pode-se destacar, principalmente: (i) o alto custo dos sensores sem fio atualmente (*hardware*) e (ii) complexidade da instalação e manutenção dos ambientes de desenvolvimento. Por exemplo, os sensores MICAz da Memsic [Su and Alzaghali 2008] necessitam de um sistema operacional rudimentar, como o TinyOS [Hill et al. 2000, Delicato 2005]. Para instalação do TinyOS é necessária a configuração do ambiente de desenvolvimento na máquina do usuário, que é um procedimento complexo.

O objetivo deste trabalho é prover uma solução onde desenvolvedores evitem executar complexas instalações e atualizações em seus computadores pessoais ou em laboratórios. Com isso, acelerar o aprendizado do desenvolvimento de aplicações para RSSF.

Nesse contexto é apresentado a Web2Compile, uma WebIDE que permite o desenvolvimento de aplicações para RSSF através de um navegador de internet. O objetivo de uma WebIDE é transformar a IDE (*Integrated Development Environment*) tradicional em um conjunto de serviços acessíveis integrados, por intermédio de um navegador web, e enriquecido com as tecnologias da Web 2.0 [Murugesan 2007].

O diferencial da Web2Compile é permitir a todos os usuários desenvolver aplicações para RSSF com uma interface simples bastando dispor de um navegador de internet atualizado. A ferramenta desenvolvida destina-se a usuários que: (i) não tem tempo ou experiência para realizar uma configuração de ambiente (como por exemplo, em sala de aula), (ii) não tem um bom enlace de dados para realizar o download do sistema operacional, (iii) não possui um dispositivo sensor, como o *hardware* sensor MICAz.

O restante deste artigo está organizado da seguinte forma. Na seção 2 são destacados os principais trabalhos relacionados. Na seção 3 é apresentada a ferramenta proposta (Web2Compile) e a sua arquitetura de software. Na seção 4 são apresentados os experimentos realizados e a análise dos resultados obtidos nos testes. Na seção 5 é apresentada uma breve descrição do demo que será apresentado. Finalmente, na seção 6 são tecidas as considerações finais e as propostas de trabalhos futuros.

2. Trabalhos Relacionados

Aplicações desenvolvidas e que seguem o paradigma da web 2.0, como é o caso das WebIDEs, já vem sendo desenvolvidas a algum tempo. A grande maioria das WebIDEs possui o único objetivo de facilitar a vida do desenvolvedor e dentre as particularidades mais interessantes, propõem-se a disponibilizar: (i) IDE, com destaque na sintaxe da linguagem particular; (ii) compilador interno para execução do código online; (iii) servidor com autenticação para usuários armazenarem seus programas para uso posterior. Dentre estas ferramentas usufruídas através de navegadores de internet, podemos destacar Arvue [Aho et al. 2011], WWWorkspace [Ryan 2007], eLuaproject e eXo Cloud IDE.

No Arvue [Aho et al. 2011] permite-se o desenvolvimento simples, publicação de aplicações destinadas à web e a hospedagem como um serviço. Implementações feitas com a aplicação são criadas no navegador utilizando uma interface de projeto (*interface designer*) e um editor de código integrado. Os desenvolvimentos são armazenados em um sistema de controle de versões fornecidas pelo próprio Arvue e que podem ser publicadas na nuvem (*cloud*). A ferramenta, foi desenvolvida utilizando-se para implementação um *framework* de código aberto para o desenvolvimento de Java Servlet baseado em aplicações web, denominado Vaadin. Segundo [Aho et al. 2011], sua intenção foi apresentar um editor útil para uma única tarefa: criação e publicação de aplicativos Vaadin de pequeno porte para *iPhone* baseado na web. Diferentemente da estrutura de desenvolvimento Arvue, este trabalho não tem o perfil de prover uma interface de desenvolvimento, mas semelhantemente, geram-se para o download no sensor (*hardware*), programas codificados obtidos através da saída da compilação. Vale ressaltar que Arvue é uma implementação não direcionada para RSSF.

WWWorkspace [Ryan 2007] é baseado em um ambiente de desenvolvimento integrado Java voltado para web construído em cima do Eclipse. A IDE do aplicativo permite que seus usuários carreguem seu espaço de trabalho e de código a partir de qualquer computador tradicional com um navegador de internet disponível. Vale notar que diferente das

outras ferramentas, para utilização do WWWorkspace, o usuário deve fazer o download do programa no site do desenvolvedor, que contém todos os arquivos necessários para executar o aplicativo, incluindo um servidor Jetty, plug-ins Eclipse e biblioteca DOJO javascript. Diferentemente, neste trabalho, ao contrário do WWWorkspace, possui-se um servidor atuando diretamente como compilador das implementações e este exige o usuário de fazer downloads de plug-ins da internet. Apoiado na mesma filosofia da ferramenta WWWorkspace, este trabalho também se diferencia por estar relacionado estritamente ao desenvolvimento de aplicações para RSSF.

O Projeto eLua (*eLua Project*) tem como objetivo introduzir a linguagem de programação Lua para o mundo do desenvolvimento de software embarcado. Lua é o exemplo perfeito de uma linguagem minimalista, mas totalmente funcional. Embora geralmente anunciada como uma "linguagem de script" e usada em conformidade especialmente na indústria do jogo, ela também é plenamente capaz de executar programas independentes. Sua pequena exigência de recursos torna a linguagem Lua adequada para um grande número de famílias de microcontroladores. Assim como outras WebIDE o objetivo do projeto é ter um ambiente de desenvolvimento totalmente funcional no próprio microcontrolador, sem a necessidade de instalar um conjunto de ferramentas específicas e configurações de ambiente de programação. Diferentemente, este trabalho não tem o objetivo de desenvolver aplicações para aplicativos embarcados, somente para RSSF.

eXo Cloud IDE é um ambiente de desenvolvimento baseado na web que permite o desenvolvimento colaborativo de aplicações que podem ser implantadas diretamente em um Ambiente Heroku PaaS (*Plataform as a Service*) [Beimborn et al. 2011]. Heroku é uma plataforma baseada na Web, muito poderosa para as linguagens Ruby, JavaScript, incluindo o Node.js e recentemente, foi incluído o apoio a aplicações Java voltadas para web. A implantação diretamente dentro de um ambiente PaaS permite migração rápida a partir da fase de desenvolvimento para a implantação. eXo Cloud IDE também contém um editor em tempo real de colaboração para utilização com até cinco pessoas. A eXo Cloud não possui o objetivo de desenvolver aplicações para RSSF.

3. Web2Compile

A Web2Compile é uma WebIDE (código disponível com documentação em <http://www.labnet.nce.ufrj.br/web2compile.html>), voltada para o desenvolvimento de aplicações para RSSFs. A arquitetura física deste projeto pode ser vista na Figura 1. Seguindo a arquitetura cliente-servidor e baseado na web como visto em [Chen and Teng 2002] o esquema arquitetural consiste nos seguintes componentes: (i) computadores clientes; e (ii) e um servidor web.

No servidor Web está hospedado o sistema operacional para RSSF. No servidor web, o ambiente de desenvolvimento para RSSF está configurado para executar simulações e compilações. O usuário terá opção de efetuar o desenvolvimento da sua aplicação na máquina cliente ou na própria página do sistema, como pode ser visto na Figura 1. Optando-se pelo desenvolvimento na máquina cliente, o usuário poderá escolher por fazer o *upload* dos arquivos necessários dependendo da aplicação que deseja.

Os dispositivos clientes tem a principal função de fazer o carregamento (*upload*) do código para o servidor web dedicado. Nesta primeira parte do processo, como pode ser visto na Figura 2, o cliente tem a possibilidade de escolher dentre duas possibilidades

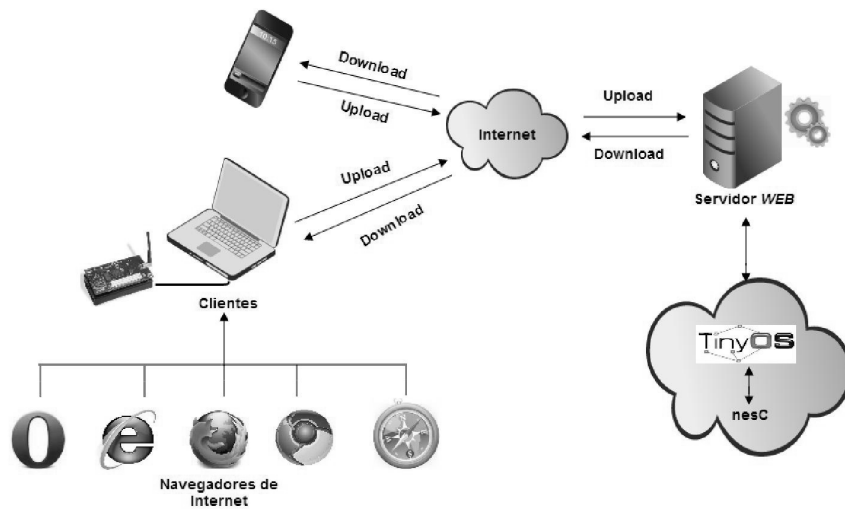


Figura 1. Arquitetura Física

de resposta para sua compilação: (i) somente simulação; ou (ii) download do código compilado.

O código será executado logo após que todos os arquivos necessários à compilação, sejam enviados (*upload*) ao compilador localizado no servidor web. Após isto, gerar-se-ão as imagens necessárias para que seja possível realizar o envio para o *hardware* do sensor.

A sequência de funcionamento da Web2Compile se dá da seguinte maneira: a partir da página carregada em seu navegador de internet, o usuário poderá fazer a seleção dos arquivos de acordo com as duas opções disponibilizadas pela aplicação, que são: (i) Somente a simulação; ou (ii) Somente o *deploy*.

Optando-se pela simulação, deverão ser selecionados os arquivos necessários e posteriormente, estes deverão ser carregados (*upload*) para o compilador hospedado no servidor. No momento da compilação, se todos os arquivos selecionados estiverem corretamente implementados de acordo com seu desenvolvimento, haverá sucesso na compilação e estes posteriormente, deverão ser disponibilizados para download pela própria aplicação, a partir de um botão indicador de download. Em contrapartida, se não houver sucesso na compilação, novamente, será necessário revisar a implementação desenvolvida para que seja feita a correção dos erros encontrados pelo compilador e, por conseguinte, realizar a seleção dos arquivos a serem compilados. Optando-se pelo *deploy*, deve-se fazer a seleção de arquivos necessários ao mesmo. Neste caso, os arquivos selecionados passarão pelo mesmo processo supracitado no caso da Simulação. No entanto, neste caso com o sucesso na compilação, gerar-se-á a construção (*build*) da imagem dos arquivos. Estes arquivos gerados, poderão ser descarregados (*download*), assim como a simulação, para o computador. A única diferença é que a imagem gerada a partir da construção do compilador poderá ser carregada para o sensor, a partir da configuração correta do *hardware* no computador.

No que diz respeito a arquitetura lógica, o Web2Compile é composto de 3 classes. A classe Gerente se que controla a página principal e recebe todas as requisições feitas

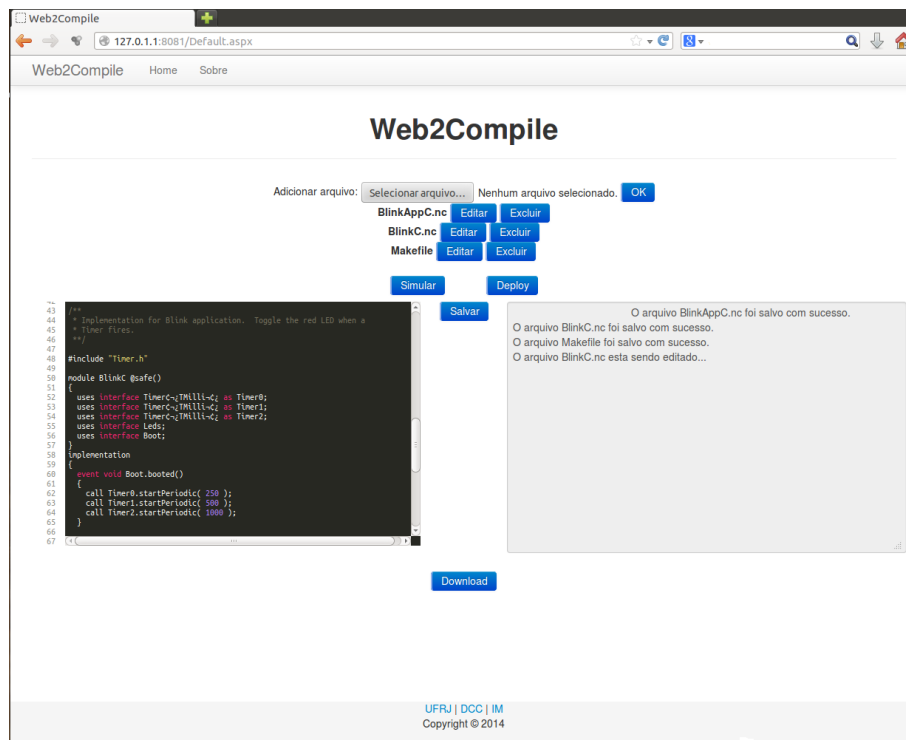


Figura 2. Interface Web2Compile

por ela. A classe `ArquivoBO` é a classe responsável por ações de pastas e arquivos. Por fim a classe `Sensor` é a responsável por interagir com o sistema Operacional TinyOS.

4. Implementação e Testes

Nesta seção são descritos: (i) as tecnologias utilizadas, assim como as ferramentas de desenvolvimento da WebIDE Web2Compile e os *frameworks*; (ii) as métricas utilizadas para avaliar o sistema e (iii) finalmente descrevem-se os testes de escalabilidade.

4.1. Tecnologias utilizadas na Implementação do Web2Compile

O objetivo do sistema Web2Compile consiste no provimento de um sistema baseado na web, de modo que sua arquitetura é baseada no modelo cliente-servidor. Optou-se pela linguagem de programação C#, utilizando a IDE MonoDevelop, versão 2.8.0, e o sistema Operacional Debian 7.0, versão *Wheezy*. Os testes foram realizados com execução a partir de um servidor com processador Intel i5-2500k, quatro núcleos (*quad-core*) a 3.3 Ghz e 8Gb de memória RAM.

Atualmente no protótipo desenvolvido, o Web2Compile suporta aplicações desenvolvidas na linguagem de programação nesC [David et al. 2003], para o sistema operacional TinyOS [Hill et al. 2000]. Para a edição de código dentro da plataforma, utilizou-se a ferramenta ACE (<http://ace.c9.io/#nav=about>).

Para a simulação, utilizamos o TOSSIM [David et al. 2003], um simulador de eventos discretos, desenvolvido para a simulação do TinyOS cuja arquitetura é baseada em componentes para RSSF. TOSSIM vêm com as características do microcontrolador AVR ATmega128 pré-programadas no simulador para atuar como uma abstração de *hard-*

ware virtual para o sensor MICAZ, utilizado neste projeto. O TOSSIM realiza a construção diretamente dos componentes do TinyOS e tem a capacidade de fazer a ligação de componentes, utilizando todas as interfaces necessárias.

4.2. Métricas

Foram utilizadas as seguintes métricas de medição: (i) Número de conexões simultâneas: quantidade de acessos recebidos pelo servidor simultaneamente para a execução de um processo (simulação ou compilação); (ii) Tempo médio, a média do tempo que leva para executar as requisições de um número “X” de conexões simultâneas; (iii) Memória consumida: quantidade de memória RAM alocada para a execução de “X” conexões simultâneas e (iv) Taxa de processamento: porcentagem disponibilizada pela CPU para a execução de um ou mais processos simultâneos.

4.3. Testes de Escalabilidade

Os Testes de Escalabilidade foram utilizados com o intuito de verificar se o Web2compile mantém o desempenho dado o aumento da quantidade de conexões simultâneas de simulações da aplicação Blink. Foram utilizadas 1, 3, 5, 7 e 10 conexões simultâneas, onde cada computador está conectado ao servidor através de um switch.

Em relação ao tempo médio, em segundos, de uma execução em cada situação anteriormente apresentada, podemos observar na Figura 3, abaixo que, embora quando exista apenas uma execução, o tempo médio foi bem abaixo dos demais. Em relação à execução de um processo com três execuções simultâneas, foram 2,5 segundos mais rápido. No geral, os tempos de execução com 3, 5, 7 e 10 conexões simultâneas apresentaram, em média, uma aumento de 0,5 segundos (no tempo de execução, de um para outro). Com isso, pode-se concluir que, mesmo com um o número de conexões simultâneas elevando-se, o tempo de conclusão da simulação não teve uma variação significativa.

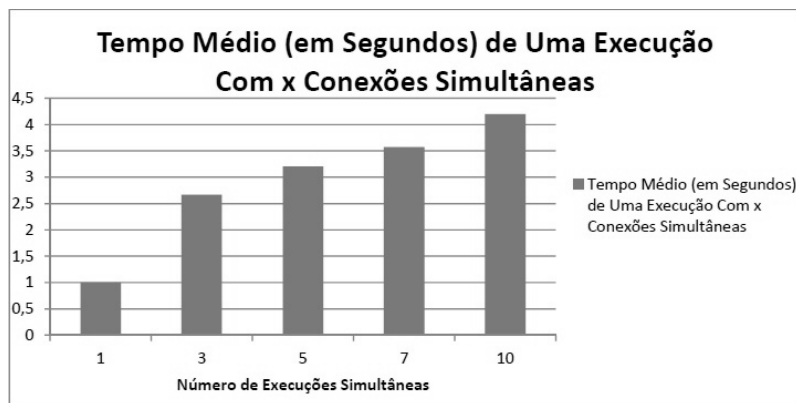


Figura 3. Tempo Médio (em segundos) de uma execução

Agora, em relação a porcentagem de utilização da CPU em relação ao mesmo número de conexões simultâneas, observou-se que a utilização da CPU aumentou de forma linear, como esperado, de acordo com o aumento das simulações simultâneas. Vale salientar que todas as *threads* foram criadas em um mesmo núcleo do processador, por isso, a elevada taxa de utilização do núcleo do processador. Este fato decorre da não sobrecarga do servidor se, em algum momento, caso ocorra um elevado número de conexões

simultâneas, faça-se com que o núcleo utilizado para processamento das threads fique alternando entre as mesmas. Em decorrência disso, observou-se que, com uma conexão apenas, o processador alocando requisito apenas para uma conexão, a taxa de processamento chegou a 40% da capacidade de um núcleo e, em decorrência da elevação das conexões simultâneas, até 10 conexões, a taxa de processamento alcançou 100% do núcleo, conforme pode ser visto na Figura 4.

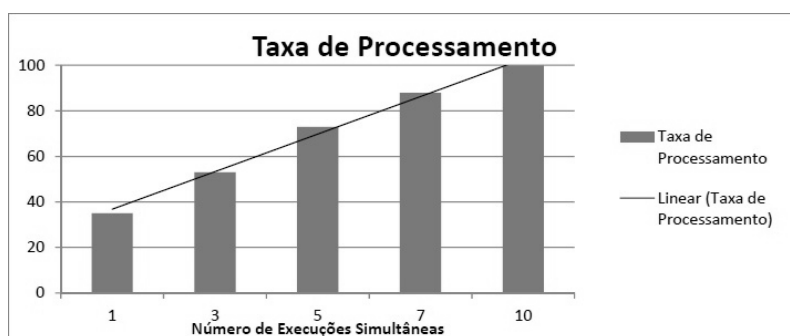


Figura 4. Taxa de Processamento

Com relação ao consumo de memória, pode-se observar que no teste realizado com uma única conexão apenas, o consumo alcançou 10 Mb de memória RAM para a execução da simulação pelo TOSSIM, devido aos processos necessários para realização da mesma. Quase linearmente, como esperado, a quantidade de memória consumida foi aumentando, até o momento que, em 10 execuções simultâneas, alcançou-se a quantidade de 122 Mb de memória RAM.

5. Descrição da Demonstração

A demonstração do sistema será feita com a construção de uma aplicação de monitoramento da temperatura do ambiente formada por dez dispositivos de comunicação Micaz, ligados a uma estação base. Os sensores coletarão os dados a cada 5 segundos e enviarão para a estação base onde será feita a média aritmética dos dados coletados. Essa aplicação será construída dentro da própria WebIDE. Será realizada então a implantação em Sensores reais (nós micaz) e no ambiente simulado com o TOSSIM.

Além disso, também estarão disponíveis outras aplicações padrões do TinyOS para a instalação e teste da plataforma. Ainda estará disponível uma máquina para que os conferencistas possam desenvolver sua própria aplicação dentro da IDE.

6. Conclusão

Este trabalho apresentou uma proposta de fomento ao desenvolvimento de aplicações relacionadas às RSSF e visa incentivar novos desenvolvedores e entusiastas ao desenvolvimento de aplicações para a tecnologia. No sistema Web2Compile, é estritamente desnecessário o *download* de aplicações para instalação do ambiente de desenvolvimento, muitas vezes complexas, além de configurações de ambientes em sistemas operacionais, onde se demanda experiência para que se tenha uma ferramenta apta ao desenvolvimento de aplicações para RSSF. Vale ressaltar também que este trabalho, exige ao usuário, fazer a compilação de seus códigos em IDEs locais.

Quanto aos trabalhos futuros, pode-se estudar mais profundamente aplicações de sistemas vislumbradas a partir da temática das RSSF e das WebIDEs. Algumas aplicações inerentes ao tema que podem vir a ser fomentadas como estudo são: (i) implementar o *Remote Flashing*, que possibilita que a imagem seja carregada em um sensor conectado a uma porta usb de um computador tradicional através do browser; (ii) utilizar outros sistemas operacionais para RSSF como o Contiki [Dunkels et al. 2006] e (iii) inserir outros tipos de ambiente de simulação que não o TOSSIM.

Referências

- Aho, T., Ashraf, A., Englund, M., Katajamäki, J., Koskinen, J., Lautamäki, J., Nieminen, A., Porres, I., and Turunen, I. (2011). Designing ide as a service. *Communications of Cloud Software*, 1(1).
- Beimborn, D., Miletzki, T., and Wenzel, S. (2011). Platform as a service (paas). *Business & Information Systems Engineering*, 3(6):381–384.
- Chen, P. M. and Teng, W.-G. (2002). Collaborative client-server architectures in the web-based viewing scheme. *Sheraton Waikiki Hotel Honolulu, Hawaii, USA*.
- Culler, D., Estrin, D., and Srivastava, M. (2004). Guest editors’ introduction: overview of sensor networks. *Computer*, 37(8):41–49.
- David, G., Philip, L., David, C., and Eric, B. (2003). Nesc 1.1 language reference manual. *Chicago, Mayo*.
- Delicato, F. C. (2005). Middleware baseado em serviços para redes de sensores sem fio. *PhD Thesis, Federal University of Rio de Janeiro, Brazil*.
- Dunkels, A. et al. (2006). The contiki operating system. *Web page. Visited Oct, 24*.
- Hill, J., Szewczyk, R., Woo, A., Hollar, S., Culler, D., and Pister, K. (2000). System architecture directions for networked sensors. In *ACM SIGOPS operating systems review*, volume 34, pages 93–104. ACM.
- Loureiro, A. A., Nogueira, J. M. S., Ruiz, L. B., de Freitas Mini, R. A., Nakamura, E. F., and Figueiredo, C. M. S. (2003). Redes de sensores sem fio. In *Simpósio Brasileiro de Redes de Computadores (SBRC)*, pages 179–226.
- Murugesan, S. (2007). Understanding web 2.0. *IT professional*, 9(4):34–41.
- Ryan, W. (2007). Web-based java integrated development environment. *BEng thesis, University of Edinburgh*.
- Su, W. and Alzaghal, M. (2008). Channel propagation measurement and simulation of micaz mote. *W. Trans. on Comp.*, 7(4):259–264.